

ローカル LLM × 音楽生成システム

ABC Music Suite

技術文書 v2.0

システム設計・構成・API仕様・セットアップガイド



ABC Music Suite



目次

1	システム概要	3
1.1	主な機能	3
2	システムアーキテクチャ	4
2.1	全体システム構成図	4
2.2	ポート割り当て一覧	4
2.3	主要データフロー	4
2.3.1	自動作曲シーケンス	5
2.3.2	MIDI・ABC 相互変換シーケンス	5
3	環境構築とセットアップ	6
3.1	前提条件・必要環境	6
3.2	依存関係のインストール	6
3.2.1	1. Python バックエンドの依存関係	6
3.2.2	2. Node.js MIDI サーバーの依存関係	7
3.2.3	3. フロントエンド (React) の依存関係	7
3.3	起動手順	7
3.3.1	一括起動 (推奨)	7
3.3.2	個別起動 (開発・デバッグ用)	7
3.4	起動後のアクセス先 URL	8
3.5	設定の変更	8
3.5.1	Ollama サーバーの IP アドレス設定	8
4	システムの利用方法	9
4.1	 自動作曲タブ	9
4.2	 ハーモニー付与タブ	9
4.3	MIDI ↔ ABC 変換タブ	9
4.4	 AI チャットタブ	9
5	API リファレンス	10
5.1	システム・ステータス系 API	10
5.1.1	GET /api/health	10
5.1.2	GET /api/models	11
5.2	自動作曲・ハーモニー付与 API	11
5.2.1	POST /api/compose	11
5.2.2	POST /api/harmonize	12
5.3	MIDI 変換・編集 API	12
5.3.1	POST /api/midi-to-abc	12
5.3.2	POST /api/abc-to-midi	13
5.4	チャット API	13

5.4.1	POST /api/chat	13
6	Raspberry Pi 連携ガイド	14
6.1	セットアップ手順	14
6.1.1	1. 関連ファイルのコピー	14
6.1.2	2. ラズパイ側での環境構築	14
6.1.3	3. ホスト PC の IP アドレス設定	14
6.1.4	4. ホスト PC での外部アクセス許可	15
6.2	接続のテスト	15
6.3	利用方法	15
6.3.1	CUI チャットの実行	15
6.3.2	ABC 楽譜ファイルの直接演奏	15
6.4	エイリアス一覧	16
6.5	トラブルシューティング	16
7	開発者向け設計詳細と拡張性	16
7.1	疎結合設計の原則	17
7.2	モジュール依存関係図	17
8	ライセンスと品質保証・免責事項	18
8.1	製品保証とサポートの範囲	18
8.1.1	1. 動作保証環境	18
8.1.2	2. 生成物（音楽データおよびテキスト）に関する責任	18
8.2	責任の制限（免責事項）	18
8.3	ライセンス規定	19

1 システム概要

ABC Music Suite は、ローカルマシン上で動作する大規模言語モデル（LLM）である「Ollama」を活用し、音楽の自動作曲、ハーモニー（伴奏）の付与、および楽譜・音源データの変換と管理を一元的に行うことができる統合システムです。

外部のクラウド API キー（ChatGPT や Claude など）を一切必要とせず、すべての AI 推論プロセスがローカル環境で完結するため、機密性の高いオフライン環境や、通信帯域に制限のある環境（Raspberry Pi など）でも安定して動作します。

1.1 主な機能

本システムは、音楽制作と AI インタクションをサポートする 5 つのコア機能を提供します。それぞれの機能の詳細は表 1 の通りです。

表 1 ABC Music Suite の主要機能一覧

機能名	説明
 自動作曲	ユーザーが入力したテーマ（例：「爽やかな朝の森」）や音楽的パラメータ（キー、テンポ）に基づいて、AI が ABC 記法の楽譜を自動生成し、即座に MIDI および WAV 音声としてレンダリング・再生します。
 ハーモニー付与	単旋律（メロディのみ）の ABC 楽譜に対して、AI が自動的に適切な伴奏コード（和音）や追加のパートを構成し、ハーモニー豊かな楽曲へとアップグレードします。
MIDI ↔ ABC 変換	既存の MIDI ファイルをテキストベースの ABC 記法に、またはその逆に変換します。変換エンジンには、定評あるオープンソースの abc2midi および midi2abc を使用し、高い互換性を実現しています。
 AI チャット	Ollama 上のローカル LLM と自由に対話できるチャット UI を提供します。会話中に AI が ABC 記法で音楽コードを出力した場合、システムが自動的にそれを検出し、Web 画面上で即座に楽譜表示および演奏できるようにします。
 Raspberry Pi 連携	同一ローカルネットワーク（LAN）上に存在する Raspberry Pi から、PC 上の強力な Ollama サーバーに接続し、コマンドラインベースで AI チャットや自動演奏を実行します。

ローカルファーストのメリット

本システムはローカル LLM（デフォルトでは qwen3.5:9b を推奨）をベースに設計されています。これにより、インターネット接続が不要となり、API の利用料金や利用制限を気にすることなく、無限に音楽の作成・実験を行うことができます。また、PC に搭載された GPU（NVIDIA 製 GeForce など）を最大限に活用することで、数秒～十数秒での高速な楽譜生成が可能です。

2 システムアーキテクチャ

本システムは、機能ごとの責務を明確に分離した「マルチレイヤー（多層）アーキテクチャ」を採用しています。各コンポーネント間は疎結合に設計されており、API（HTTP/SSE）および明確なインターフェースを介してのみ通信を行います。

2.1 全体システム構成図

システム全体の構造および各サブシステムの通信関係を図 1 に示します。

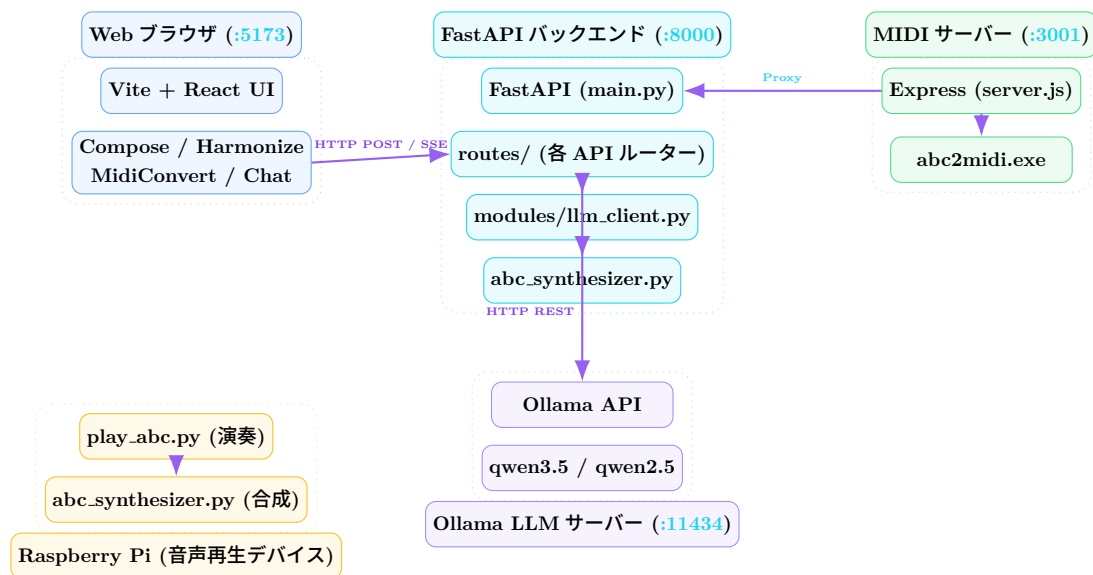


図 1 ABC Music Suite システムアーキテクチャ全体図

2.2 ポート割り当て一覧

システム各コンポーネントが占有するネットワークポートおよび通信プロトコルの一覧を表 2 に示します。

2.3 主要データフロー

システムの動作時におけるデータの流れをシーケンス図形式で示します。

表 2 サービスポート一覧

サービス名	標準ポート	プロトコル	役割
Vite Dev Server	:5173	HTTP	フロントエンド (React) の開発用 Web 配信
FastAPI Backend	:8000	HTTP / SSE	メイン API サービス、ファイル配信、SSE ストリーミング
MIDI Server	:3001	HTTP	abc2midi / midi2abc ラッパープロキシ
Ollama Service	:11434	HTTP	ローカル LLM 推論 API

2.3.1 自動作曲シーケンス

ユーザーが Web UI から曲のテーマを入力して作曲を指示した際、バックエンドで楽譜テキストが抽出され、音声波形（WAV）として合成されるまでの流れを図 2 に示します。

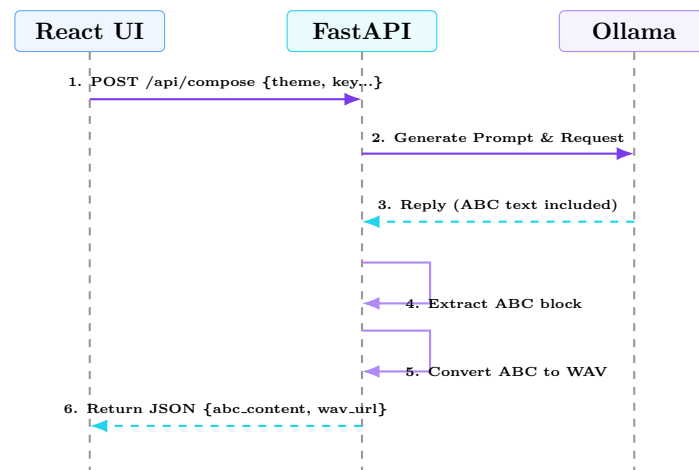


図 2 自動作曲処理のデータフロー

2.3.2 MIDI・ABC 相互変換シーケンス

MIDI ファイルが入力され、Node.js Express に配置された外部実行バイナリ `abc2midi` 等を経由して変換が実行されるフローを図 3 に示します。

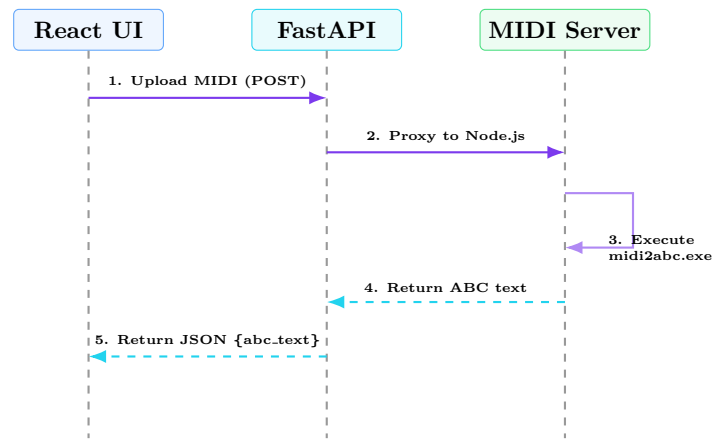


図 3 MIDI から ABC への変換フロー

3 環境構築とセットアップ

本システムは、Windows 10/11 をホスト PC とし、LAN 経由で Raspberry Pi などの外部デバイスと接続することを想定しています。以下にシステムを稼働させるためのセットアップ手順を説明します。

3.1 前提条件・必要環境

本システムを実行するには、ホスト PC に以下のソフトウェアがあらかじめインストールされている必要があります。

- **Ollama:** ローカルで大規模言語モデルを実行するエンジン。
 - 推奨モデル: qwen3.5:9b または qwen2.5-coder:7b（音楽記法（ABC 記法）の生成において高い安定性を示します）。
- **Python 3.10 以上:** バックエンドの FastAPI サーバー動作に必要。
- **Node.js 18 以上:** フロントエンドの開発・ビルドおよび MIDI サーバー動作に必要。

3.2 依存関係のインストール

リポジトリをクローンまたは展開した後、各ディレクトリで必要なパッケージをインストールします。

3.2.1 1. Python バックエンドの依存関係

コマンドプロンプトまたは PowerShell を起動し、以下のコマンドを実行します。Python バックエンドの依存パッケージインストール:

```

1 cd C:\情報科学演習\\server
2 pip install fastapi uvicorn pydantic requests

```

3.2.2 2. Node.js MIDI サーバーの依存関係

MIDI ファイルの相互変換およびラッパーとして動作する Express サーバーの依存関係をインストールします。Node.js MIDI サーバーの依存パッケージインストール:

```
1 cd C:\情報科学演習:\\midi-server
2 npm install
```

3.2.3 3. フロントエンド (React) の依存関係

Vite + React ベースの Web UI 用パッケージをインストールします。フロントエンドの依存パッケージインストール:

```
1 cd C:\情報科学演習:\\web-ui
2 npm install
```

3.3 起動手順

3.3.1 一括起動 (推奨)

本システムには、フロントエンド、バックエンド、MIDI サーバーの 3 つのサービスを一括で立ち上げるスクリプトが用意されています。

PowerShell から起動する場合は、以下のコマンドを実行します。PowerShell による一括起動コマンド:

```
1 cd C:\情報科学演習:\\
2 .\start.ps1
```

また、エクスプローラーから `start.bat` をダブルクリックすることでも、同様にすべてのサービスを一括起動できます。

3.3.2 個別起動 (開発・デバッグ用)

サービスを個別にデバッグしたい場合は、3 つのターミナルを開き、それぞれ以下のコマンドを実行します。

1. FastAPI バックエンド:

```
1 cd C:\情報科学演習:\\server
2 python -m uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

2. MIDI サーバー:


```
1 cd C:\情報科学演習\midi-server
2 node server.js
```

3. Web UI (Vite):

```
1 cd C:\情報科学演習\web-ui
2 npm run dev
```

3.4 起動後のアクセス先 URL

すべてのサービスが正常に起動すると、表 3 の URL から各サービスにアクセスできます。

表 3 アクセス先 URL 一覧

アクセス先	URL	説明
Web UI	http://localhost:5173	メインの操作画面（ここをブラウザで開きます）
FastAPI Swagger	http://localhost:8000/docs	バックエンドの API 仕様確認・テスト用インタラクティブ画面
FastAPI API	http://localhost:8000	フロントエンドやラズパイからアクセスされる API ベース
MIDI Server	http://localhost:3001	MIDI 変換を処理する Node.js のエンドポイント

3.5 設定の変更

3.5.1 Ollama サーバーの IP アドレス設定

デフォルトでは、FastAPI は同一マシン上の Ollama (<http://127.0.0.1:11434>) に接続します。Raspberry Pi など他のデバイスから接続する場合、または Ollama を別 PC で動かす場合は、`server/modules/config.py` を編集します。

`config.py` の編集例:

```
1 # Set Ollama server IP
2 OLLAMA_HOST = "192.168.0.136" # Change to your host PC's IP address
3 OLLAMA_PORT = 11434
```

また、環境変数 `OLLAMA_HOST` を設定することでも上書き可能です。

```
1 $env:OLLAMA_HOST = "192.168.0.136"
2 .\start.ps1
```

4 システムの利用方法

サービスを一括起動した後、ブラウザで <http://localhost:5173> を開くと、統合 Web UI が表示されます。UI は上部のタブで切り替えることができ、それぞれの機能を提供します。

また、UI の右上には「接続ステータスインジケータ」があり、Ollama サーバーおよび MIDI サーバーへの接続状態がリアルタイムで緑（接続中）または赤（切断）で表示されます。

4.1 🛠️ 自動作曲タブ

AI に指示を出して完全に新しい楽曲を生成させることができます。

1. **テーマの入力:** 「爽やかな朝」「哀愁漂う夜の街」など、曲のイメージをテキストで自由に入力します。
2. **パラメータ設定:**
 - **キー（調号）:** C, G, F, Am, Em などから選択します。
 - **テンポ（BPM）:** 60～200 の範囲で設定します。
3. **使用モデルの選択:** ドロップダウンから、Ollama にロードされている任意の LLM モデル（例: qwen3.5:9b）を選択します。
4. **実行:** 「作曲を開始」ボタンをクリックします。
5. **結果の確認:** 数秒後、AI が生成した楽譜テキスト（ABC 記法）とレンダリングされた楽譜が画面に表示されます。また、画面上のオーディオプレーヤーで生成された音源（WAV）をその場で再生でき、ダウンロードも可能です。

4.2 🎵 ハーモニー付与タブ

既存の単旋律（メロディのみ）に対して、AI に伴奏（コードまたは別パート）を追加させることができます。

1. **メロディ入力:** テキストエリアに既存の ABC 記法の楽譜を貼り付けます。
2. **モデルと設定の選択:** 自動作曲と同様に、モデルを設定します。
3. **実行:** 「ハーモニーを生成」ボタンをクリックします。
4. **結果の確認:** メロディラインに伴奏（コードや低音パートなど）が追加された新しい楽譜が生成されます。同様に、オーディオプレーヤーで和音を伴った演奏の再生・ダウンロードが可能です。

4.3 MIDI ↔ ABC 変換タブ

PC 上の MIDI ファイルとテキストベースの ABC 楽譜を双方向に変換できます。

4.4 💬 AI チャットタブ

Ollama のローカル LLM とリアルタイムで対話できるチャット機能です。

MIDI ↔ ABC 双方向変換の操作

■ MIDI から ABC へ変換する手順

1. ファイル選択エリアで、PC 内にある `.mid` ファイルをドラッグ&ドロップまたはクリックして選択します。
2. 「ABC へ変換」ボタンをクリックします。
3. 変換後の ABC 楽譜テキストが表示され、画面上で楽譜のプレビューおよび音源再生ができます。

■ ABC から MIDI へ変換する手順

1. エディタエリアに、直接 ABC 記法のテキストを入力または貼り付けます。
2. 「MIDI へ変換」ボタンをクリックします。
3. 変換に成功すると、生成された `.mid` ファイルのダウンロードリンクが表示されます。

- **SSE (Server-Sent Events) による逐次出力:** 回答が 1 文字ずつ流れるように表示されます。
- **楽譜の自動検出:** AI の返答の中に `x:1` から始まるような ABC 楽譜ブロックが含まれている場合、チャット UI がそれを自動的に検出し、通常のテキストとは別に「楽譜パネル」として分離して表示します。
- **即時プレビュー・演奏:** 検出された楽譜は、チャットスレッド内でそのまま `abcjs` ライブラリによって楽譜描画され、ボタン一つでブラウザのスピーカーからオーディオシンセサイザーを用いて演奏・再生することができます。

5 API リファレンス

本システムは、すべての制御を HTTP REST および Server-Sent Events (SSE) ベースの API を介して行います。バックエンド (FastAPI、ポート `:8000`) が提供する主要なエンドポイントの仕様を以下に解説します。

なお、サーバー起動後は <http://localhost:8000/docs> にアクセスすることで、Swagger UI からこれらすべての API をブラウザ上で直接対話的にテストできます。

5.1 システム・ステータス系 API

5.1.1 GET /api/health

システム全体の稼働状態および接続状況 (Ollama、Node.js MIDI サーバー) を返します。

/api/health レスポンス例:

```

1 {
2   "status": "ok",
3   "ollama": {
4     "connected": true,
5     "url": "http://127.0.0.1:11434",
6     "model_count": 3
7   },
8   "midi_server": {
9     "connected": true,
10    "url": "http://127.0.0.1:3001"
11  }
12 }

```

5.1.2 GET /api/models

Ollama サーバーに現在インストールされており、利用可能なモデルの一覧を返します。
/api/models レスポンス例:

```

1 {
2   "models": ["qwen3.5:9b", "qwen2.5-coder:7b"],
3   "base_url": "http://127.0.0.1:11434"
4 }

```

5.2 自動作曲・ハーモニー付与 API

5.2.1 POST /api/compose

指定されたテーマや音楽的パラメータに基づいて、AI（Ollama）で新規に楽曲（ABC 記法）を生成し、WAV に合成します。

表 4 /api/compose リクエストパラメータ仕様

パラメータ	データ型	デフォルト	説明
theme	string	(必須)	作曲のテーマ・イメージ（日本語可）
key	string	"C"	曲のキー（C, G, Am, Em, F, Dm など）
tempo	integer	120	テンポ BPM（60 ～ 200）
model	string	"qwen3.5:9b"	使用する Ollama 上のモデル名

/api/compose リクエスト・レスポンス例:

```

1 // リクエスト
2 {
3   "theme": "爽やかな朝の森",

```

```

4   "key": "C",
5   "tempo": 120,
6   "model": "qwen3.5:9b"
7 }
8
9 // レスポンス
10 {
11   "title": "爽やかな朝の森",
12   "abc_content": "X:1\nT爽やかな朝の森
13
14   "abc_url": "/output/compose_20260626_120000.abc",
15   "wav_url": "/output/compose_20260626_120000.wav"
16 }

```

5.2.2 POST /api/harmonize

送信された単旋律の ABC 楽譜に対して、AI が自動で伴奏コードや追加パート（ハーモニー）を付与し、WAV にレンダリングします。

/api/harmonize リクエスト例:

```

1 {
2   "melody_abc": "X:1\nT:Test\nM:4/4\nL:1/4\nK:C\nC C G G | A A G2 |",
3   "title": "twinkle_harmony",
4   "model": "qwen3.5:9b"
5 }

```

レスポンスは /api/compose と同一の形式（生成後の楽譜およびファイル URL）を返します。

5.3 MIDI 変換・編集 API

5.3.1 POST /api/midi-to-abc

アップロードされたバイナリ MIDI ファイルを解析し、ABC 記法のテキストに変換します。

- **Content-Type:** multipart/form-data
- **パラメータ:** midiFile (ファイル本体)

/api/midi-to-abc レスポンス例:

```

1 {
2   "status": "success",
3   "abc_text": "X:1\nT:Converted MIDI\nM:4/4\nL:1/8\nK:C\nC2 E2 G2 c4 |
4   ...",
5   "message": "→変換成功！MIDIABC"
6 }

```

5.3.2 POST /api/abc-to-midi

入力された ABC 楽譜テキストを MIDI ファイル ([.mid](#)) に変換します。内部的には、Node.js サーバーを経由して [abc2midi](#) エンジンが起動されます。

/api/abc-to-midi リクエスト・レスポンス例:

```

1 // リクエスト
2 {
3   "abc_text": "X:1\nT:Test\nM:4/4\nL:1/4\nK:C\nC D E F | G A B c |"
4 }
5
6 // レスポンス
7 {
8   "status": "success",
9   "message": "による変換に成功！abc2midiMIDI",
10  "midi_url": "/generated_output.mid"
11 }
```

5.4 チャット API

5.4.1 POST /api/chat

Ollama 上のモデルと対話を行うストリーミング型 API です。Server-Sent Events (SSE) を使用して、生成されたテキストをリアルタイムでクライアントにプッシュします。

/api/chat リクエスト例:

```

1 {
2   "messages": [
3     {"role": "user", "content": "メジャーで短いワルツを作ってG"}
4   ],
5   "model": "qwen3.5:9b",
6   "system_prompt": "あなたは音楽作成を支援する優秀なアシスタント"です。AI"
7 }
```

/api/chat レスポンスストリーム例（概略）:

```

1 data: {"token": "X"}
2 data: {"token": ":1\n"}
3 data: {"token": "T"}
4 ...
5 data: [DONE]
```

6 Raspberry Pi 連携ガイド

本システムは、LAN（ローカルエリアネットワーク）内に存在する Raspberry Pi から、強力な GPU を搭載したホスト PC の Ollama（LLM サーバー）と接続テストを行ったり、ローカルで楽譜ファイルや ABC 記法から直接音楽を波形合成して演奏を行う機能を提供しています。

6.1 セットアップ手順

以下の手順に沿って、ホスト PC および Raspberry Pi の設定を行います。

6.1.1 1. 関連ファイルのコピー

ホスト PC の PowerShell から、Raspberry Pi 用のクライアントスクリプト群を転送します。転送には専用の自動化スクリプト `copy-to-pi.ps1` を利用できます。**コピー実行コマンド**:

```
1 cd C:\practice
2 .\copy-to-pi.ps1
```

または、手動で `scp` を使用してコピーすることも可能です。

```
1 scp C:\practice\raspberry\* pi@<raspi_ip>:~/llm-client/
```

6.1.2 2. ラズパイ側での環境構築

Raspberry Pi に SSH 等でログインし、セットアップスクリプトを実行してパッケージとエイリアスを設定します。**環境構築スクリプト実行**:

```
1 cd ~/llm-client
2 bash setup.sh
3 source ~/.bashrc
```

6.1.3 3. ホスト PC の IP アドレス設定

Raspberry Pi 上の `~/llm-client/config.py` をテキストエディタ（nano など）で開き、ホスト PC の実際の IP アドレスを指定します。**接続先設定 (config.py)**:

```
1 # ~/llm-client/config.py
2 OLLAMA_HOST = "192.168.0.136" # Change to your host PC's IP address
3 OLLAMA_PORT = 11434
4 DEFAULT_MODEL = "qwen3.5:9b"
```

ホスト PC の IP アドレスを確認する方法

ホスト PC の PowerShell で以下のコマンドを実行することで、自身の IPv4 アドレスを確認できます。

```
1 (Get-NetIPAddress -AddressFamily IPv4 | Where-Object
   {$_ .InterfaceAlias -ne "Loopback*"}).IPAddress
```

6.1.4 4. ホスト PC での外部アクセス許可

デフォルトでは、Ollama はセキュリティ上の理由から localhost (127.0.0.1) からの接続のみを許可しています。外部の Raspberry Pi からアクセスを受け付けるには、環境変数を設定して Ollama を起動し直す必要があります。

Ollama 公開設定:

```
1 $env:OLLAMA_HOST = "0.0.0.0"
2 ollama serve
```

6.2 接続のテスト

設定が完了したら、Raspberry Pi 上で接続疎通テストスクリプトを実行します。

```
1 python3 ~/llm-client/test_connection.py
```

接続が成功すると、ホスト PC 上の Ollama で動作している利用可能モデルの一覧が表示されます。

6.3 利用方法

6.3.1 ABC 楽譜ファイルの直接演奏

ローカルに保存されている .abc 楽譜ファイル、または直接 ABC テキストを指定してラズパイで自動演奏させることができます。

```
1 play-abc /path/to/score.abc
```

6.4 エイリアス一覧

setup.sh を実行すると、Raspberry Pi の ~/.bashrc に以下のショートカットコマンドが追加され、簡単に操作できるようになります。

表 5 Raspberry Pi エイリアスコマンド

コマンド	実際の実行コード	役割
<code>llm-test</code>	<code>python3 ~/llm-client/test_connection.py</code>	ホスト PC との接続テスト
<code>play-abc</code>	<code>python3 ~/llm-client/play_abc.py</code>	指定した ABC 楽譜の演奏

6.5 トラブルシューティング

接続がタイムアウトまたは拒否される場合

1. **ネットワークの確認**: ホスト PC と Raspberry Pi が同じルーター（Wi-Fi 等）に接続されており、相互に ping が通るか確認してください。
2. **Windows ファイアウォール**: Windows 側でポート `:11434` へのインバウンド接続がブロックされている可能性があります。PowerShell から以下のコマンドを実行してポートを解放してください。

```
1 New-NetFirewallRule -DisplayName "Ollama" -Direction Inbound
   -Protocol TCP -LocalPort 11434 -Action Allow
```

3. **Listen アドレス確認**: ホスト PC で `netstat -an | findstr 11434` を実行し、ローカルアドレスが `0.0.0.0:11434` または `*:11434` で LISTEN 状態になっているか確認してください。

音が出ない (Raspberry Pi)

スピーカーから音声波形を出力するには、オーディオユーティリティ `aplay` がインストールされている必要があります。

```
1 sudo apt-get update
2 sudo apt-get install alsa-utils
```

7 開発者向け設計詳細と拡張性

本システムは、将来の機能追加や LLM プロバイダの変更、UI フレームワークの刷新に耐えうるよう、疎結合なモジュール設計を徹底しています。

7.1 疎結合設計の原則

本システムにおける各モジュールの結合は、HTTP API (REST / SSE) または明確な Python クラスメソッドの引数/戻り値に限定されています。これにより、以下の変更が生じた場合でも、他のコンポーネントに影響を与えることなく改修が可能です。

1. **LLM (AI) プロバイダの変更**: Ollama から外部 API (OpenAI, Anthropic など) に移行する場合でも、`server/modules/llm_client.py` 内の `OllamaClient` クラス（またはこれ

に代わる共通インターフェース) の内部処理を書き換えるだけで、FastAPI のルーティングやフロントエンド側には一切の変更が発生しません。

2. **フロントエンドの刷新:** 現在の Vite + React から他のフレームワーク (Next.js や Vue.js など) へ移行、あるいはネイティブアプリ (iOS / Android) を作成する場合でも、バックエンドの OpenAPI (FastAPI) が提供するエンドポイント群 (:8000) は同一であるため、バックエンドコードの変更は一切不要です。
3. **MIDI 変換エンジンの変更:** 現在は C++ で書かれた実行バイナリ `abc2midi` を Node.js でラップして処理していますが、将来的に Python のネイティブライブラリによる処理へ移行する場合、`server/routes/midi_convert.py` を書き換えて Node.js サーバーへのプロキシを解除するだけで、フロントエンド側の変換インターフェースは維持されます。

7.2 モジュール依存関係図

フロントエンドおよびバックエンドにおけるコードモジュール同士の依存関係を図 4 に示します。

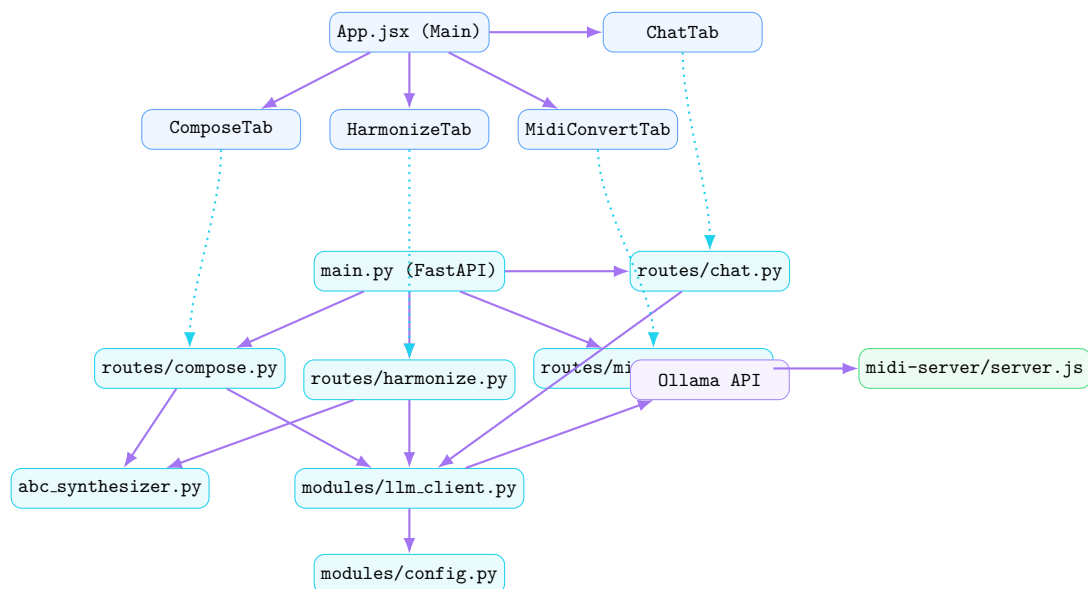


図 4 フロントエンド・バックエンドにおけるモジュール依存関係

8 ライセンスと品質保証・免責事項

本章では、開発元 (以下「メーカー」) が提供する本システム (ABC Music Suite) の利用条件、メーカーが負う責任範囲、および保証内容について規定します。本システムを使用することにより、利用者は本規定に同意したものとみなされます。

8.1 製品保証とサポートの範囲

本システムは、特定の開発・評価用プロトタイプとして設計されています。メーカーは、本システムが利用者の特定の目的に適合すること、または動作に中断やエラーが生じないこと

について、明示的にも黙示的にも保証いたしません。

8.1.1 1. 動作保証環境

メーカーが動作を確認し、サポート対象とする基本環境は以下の通りです。これら以外の環境（独自の OS カスタム、サポート対象外の Python バージョンなど）における不具合については、メーカーは対応いたしかねます。

- **ホスト PC:** Windows 10/11 (64bit), Python 3.10 以上, Node.js 18 以上
- **Ollama:** 推奨モデル qwen3.5:9b および qwen2.5-coder:7b
- **Raspberry Pi:** Raspberry Pi OS (Bullseye/Bookworm), Python 3.9 以上, ALSA 音響ユーティリティが正常にインストールされていること

8.1.2 2. 生成物（音楽データおよびテキスト）に関する責任

本システムで生成される ABC 楽譜データおよび WAV 音源は、すべてローカル LLM（AI）による推論の帰結です。

- **出力の不確実性:** AI の性質上、常に音楽的に正しい結果が出力されるとは限りません。生成された楽譜における不協和音、音割れ、演奏不可能なフレーズの出現について、メーカーは修正の義務を負いません。
- **権利関係:** 生成された楽曲が第三者の著作権またはその他の権利を侵害しないことについて、メーカーは保証しません。生成物の商業利用や公開に際しては、利用者の自己責任において法的確認を行ってください。

8.2 責任の制限（免責事項）

本ソフトウェアの使用または使用不能から生じるすべての直接的、間接的、付随的、特別または結果的な損害（データの損失、業務の停止、ラズパイ等のハードウェア故障、ネットワーク帯域の枯渇、不正アクセス等を含む）について、メーカーおよび開発者は、契約上の請求、不法行為（過失を含む）、またはその他の法的根拠のいずれによるかを問わず、一切の責任を負いません。

セキュリティに関するメーカーからの警告

ホスト PC 側で Ollama サーバーを外部公開設定（OLLAMA_HOST=0.0.0.0）にする場合、同一 LAN 内のすべての機器から LLM サーバーへのアクセスが可能になります。公共のネットワーク（公衆 Wi-Fi など）でこの設定を行った場合、外部から不正に利用されたり、ホスト PC に負荷をかけられたりする危険性があります。外部公開は必ず信頼できるプライベートなネットワーク内でのみ実行してください。この設定に起因する不正アクセスについて、メーカーは一切の責任を負いません。

8.3 ライセンス規定

本システムのソースコードおよびドキュメントは、**MIT ライセンス**のもとで許諾されます。

MIT License

Copyright (c) 2026 ABC Music Suite 開発チーム

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.